

VITyarthi – Build Your Own Project

PROJECT REPORT

HOTEL RESERVATION SYSTEM

CSE2006 – Programming in Java

Submitted by

Name: Atharv Kala

Registration Number: 25BAI10241

School: SCAI

Academic Year: 2026

1. Introduction

The Hotel Reservation System is a command-line application developed in Java to manage common hotel operations in an organized manner. The system provides facilities for room inventory management, customer registration, booking creation and cancellation, check-in and check-out, billing, discounts, refunds, and booking history.

The project is designed to demonstrate concepts from Programming in Java, including object-oriented programming, classes and objects, encapsulation, constructors, exception handling, collections, file input/output, serialization, and date/time handling.

The application uses a menu-driven command-line interface. Data is persisted through serialized .dat files so that room, customer, and booking information can be loaded again when the application is restarted.

2. Problem Statement

Manual hotel reservation management can make it difficult to maintain room availability, customer records, booking status, cancellation refunds, and billing information consistently. A computerized reservation system is therefore required to centralize these operations and apply validation and business rules during booking and related activities.

The proposed system provides a Java-based solution that allows hotel staff to manage rooms, customers, and reservations through a single menu-driven application.

3. Functional Requirements

Module	Functionality
Room Management	Add rooms, view rooms, search rooms, view available rooms, and organize rooms by type.
Customer Management	Register customers, search customer records, view details, and maintain booking history.
Booking Management	Create, search, cancel, check in, check out, and view upcoming bookings.
Billing & Discounts	Generate bills, calculate 18% GST, apply percentage discounts, and track total amounts.
Cancellation & Refunds	Calculate refunds according to the cancellation timing and update booking status.
Statistics & Persistence	Display hotel statistics and save/load room, customer, and booking data using serialization.

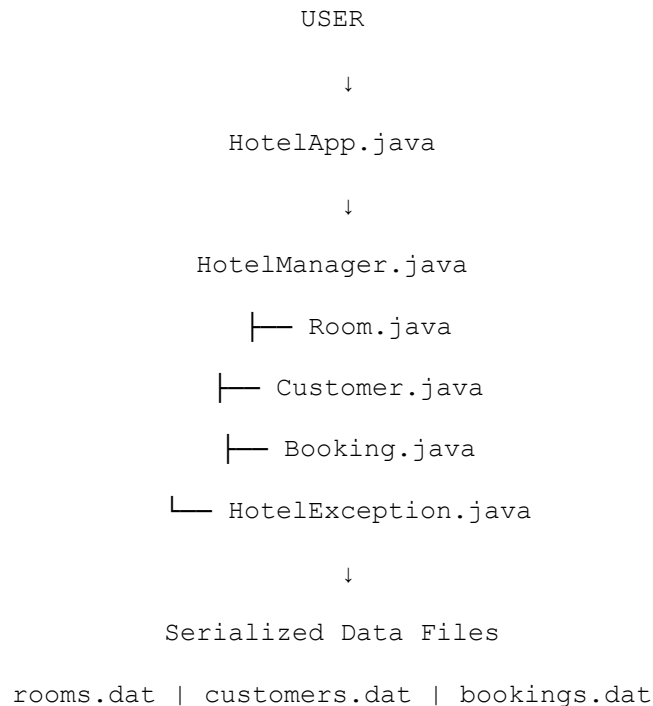
4. Non-Functional Requirements

Requirement	Description
Usability	The menu-driven CLI provides clear choices and prompts for common operations.
Reliability	Validation and exception handling are used to prevent invalid operations and handle business-rule violations.
Maintainability	The system is separated into entity classes, a manager class, a custom exception, and the main

	application.
Performance	Operations use in-memory collections and linear searches, which are appropriate for the intended small-scale application.
Data Persistence	Serialized .dat files preserve rooms, customers, and bookings between application sessions.
Resource Efficiency	The application does not require an external database and stores data locally.

5. System Architecture

The application follows a simple layered organization. HotelApp acts as the command-line interface and receives user input. HotelManager contains the main business logic and manages the Room, Customer, and Booking entities. HotelException is used for hotel-specific business errors. Data is stored through serialization.



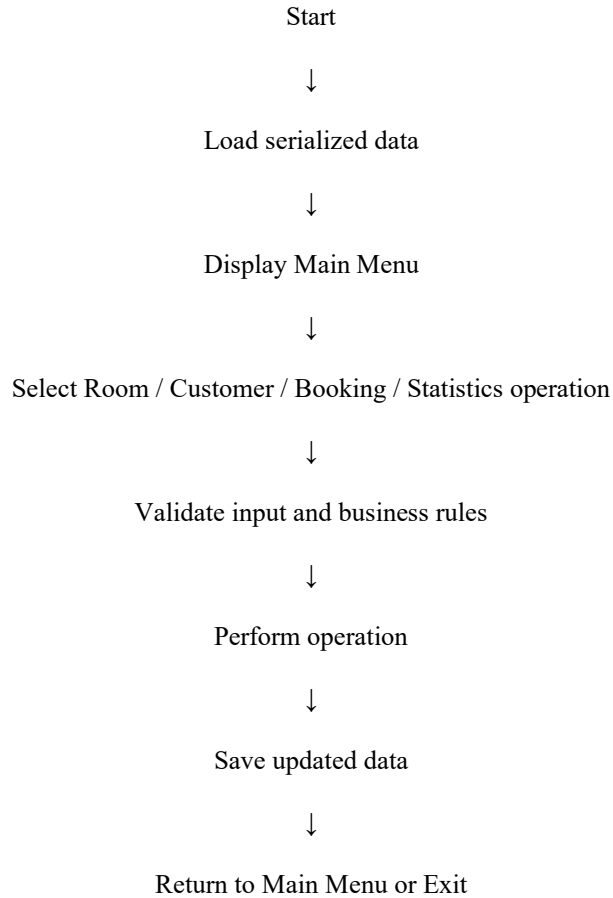
6. Design Diagrams

6.1 Use Case Diagram

Actor: Hotel Staff / Operator

- Manage Rooms → Add / View / Search / Check Availability
- Manage Customers → Register / Search / View Details
- Manage Bookings → Create / Search / Check-In / Check-Out / Cancel
- Manage Billing → Apply Discount / Generate Bill
- View Statistics → Hotel overview and upcoming bookings

6.2 Workflow Diagram



6.3 Sequence Diagram – Create Booking

- User → HotelApp: Select Create Booking
- HotelApp → User: Request booking/customer/room/dates
- User → HotelApp: Enter details
- HotelApp → HotelManager: createBooking(...)
- HotelManager → Room: Check availability
- HotelManager → Booking: Create reservation
- HotelManager → Customer: Add booking to history
- HotelManager → Data Files: Save updated records
- HotelManager → HotelApp: Success / Error
- HotelApp → User: Display result

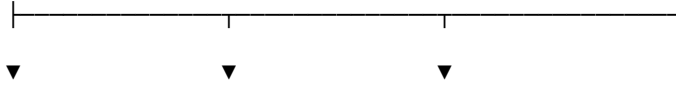
6.4 Class Diagram / Component Diagram

HotelApp

| uses



HotelManager



Room

Customer

Booking



HotelException

Room: roomNumber, roomType, price, status, amenities, capacity

Customer: customerID, name, email, phone, address, bookingHistory, totalAmountSpent

Booking: bookingID, customerID, roomNumber, dates, numberOfNights, pricing, status, discount, refund

6.5 Storage Design

The project does not use a relational database. It uses Java serialization to persist three collections of application data:

- rooms.dat – room information
- customers.dat – customer information
- bookings.dat – booking records

Logical relationships:

Customer 1 ———< Booking

Room 1 ———< Booking

Booking references one Customer and one Room.

7. Design Decisions & Rationale

Decision	Rationale
Java CLI	A command-line interface keeps the project focused on Java programming concepts and business logic.
OOP classes	Separate classes represent rooms, customers, bookings, and application management responsibilities.
ArrayList	Dynamic collections are used to store and manage application entities.
Custom Exception	HotelException provides application-specific error handling for business constraints.
Serialization	Object streams provide simple local persistence without requiring a database.
Date handling	Booking dates are parsed and used to calculate stay duration and cancellation/refund rules.
Menu-driven structure	Separate menus organize room, customer, booking, and statistics operations.

8. Implementation Details

Room.java: Represents hotel rooms and maintains room number, type, price, status, amenities, and capacity.

Customer.java: Stores customer information and an ArrayList-based booking history.

Booking.java: Represents reservations, dates, nights, pricing, status, discounts, and refunds.

HotelManager.java: Contains core business logic for rooms, customers, bookings, billing, statistics, validation, and persistence.

HotelException.java: Custom exception extending Exception for hotel-specific business errors.

HotelApp.java: Main entry point providing the interactive CLI and input handling.

The booking process validates customer and room information, checks dates and availability, calculates the number of nights, creates the booking, updates related records, and persists the data.

The cancellation policy implemented in the project provides a full refund when cancellation is more than 48 hours before check-in, a 50% refund when cancellation is within 48 hours, and no refund after check-in.

9. Screenshots / Results

- Main menu

```
C:\WINDOWS\system32\cmd.exe X Command Prompt - java Hot X + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\tanmay>cd C:\Hotel management

C:\Hotel management>javac *.java

C:\Hotel management> java HotelApp

=====
Welcome to Hotel Reservation System
=====
Loading data...

=====
HOTEL RESERVATION SYSTEM
Welcome to Grand Hotel India
=====
1. Room Operations
2. Customer Operations
3. Booking Operations
4. View Statistics
5. Exit
=====
Enter your choice: |
```

- Room management menu and available rooms

```
-----
ROOM OPERATIONS
-----
1. Add Room
2. View All Rooms
3. View Available Rooms
4. Search Room by Number
5. View Rooms by Type
6. Back to Main Menu
-----
Enter your choice: |
```

- Customer registration

```
-----
CUSTOMER OPERATIONS
-----
1. Register New Customer
2. Search Customer by ID
3. View Customer Details
4. View All Customers
5. Back to Main Menu
-----
Enter your choice: |
```

- Booking creation

```
-----  
BOOKING OPERATIONS  
-----  
1. Create Booking  
2. Search Booking by ID  
3. Check-In  
4. Check-Out  
5. Cancel Booking  
6. Apply Discount  
7. Generate Bill  
8. View All Bookings  
9. View Upcoming Bookings  
10. Back to Main Menu  
-----  
Enter your choice: |
```

- Discount application

```
-----  
BOOKING OPERATIONS  
-----  
1. Create Booking  
2. Search Booking by ID  
3. Check-In  
4. Check-Out  
5. Cancel Booking  
6. Apply Discount  
7. Generate Bill  
8. View All Bookings  
9. View Upcoming Bookings  
10. Back to Main Menu  
-----  
Enter your choice: 6  
Enter Booking ID: 1  
Enter Discount Percentage (0-100): 20  
? Discount applied successfully!  
New Total: Rs. 8000.0
```

- Generated bill

```
=====
HOTEL BILL
Grand Hotel India - Mumbai
=====

Booking ID: 1
Customer ID: 1
Room Number: 1
Check-In: 19-09-2026
Check-Out: 20-09-2026
Nights: 1
Price/Night: Rs. 10000.00
Discount Applied: Rs. 2000.00
Total Cost: Rs. 8000.00
Status: Completed
Refund Amount: Rs. 0.00
```

```
-----
Subtotal: Rs. 8000.00
Tax (18%): Rs. 1440.00
Grand Total: Rs. 9440.00
-----
```

- Check-in/check-out

BOOKING OPERATIONS

1. Create Booking
2. Search Booking by ID
3. Check-In
4. Check-Out
5. Cancel Booking
6. Apply Discount
7. Generate Bill
8. View All Bookings
9. View Upcoming Bookings
10. Back to Main Menu

```
-----
Enter your choice: 3
Enter Booking ID: 1
? Checked in successfully!

-
```

BOOKING OPERATIONS

1. Create Booking
2. Search Booking by ID
3. Check-In
4. Check-Out
5. Cancel Booking
6. Apply Discount
7. Generate Bill
8. View All Bookings
9. View Upcoming Bookings
10. Back to Main Menu

```
-----
Enter your choice: 4
Enter Booking ID: 1
? Checked out successfully!
```

10. Testing Approach

Test Case	Procedure	Expected/Reported Result
Room Management	Add/search/view rooms and verify availability changes after booking.	Pass
Booking & Refund	Create a five-day booking and verify refund calculation for different cancellation timings.	Pass
Discount & Billing	Apply a 20% discount to a Rs. 10,000 booking and verify 18% GST billing.	Pass
Customer Workflow	Register a customer, create bookings, view history, and track spending.	Pass
Input Validation	Test invalid menu choices, data types, and date formats.	Pass

The project documentation specifies compilation using `javac *.java` and execution using `java HotelApp`. The application creates the serialized data files automatically during use.

11. Challenges Faced

- Managing relationships between rooms, customers, and bookings while keeping records consistent.
- Validating booking dates and preventing overlapping reservations.
- Implementing cancellation rules and calculating refunds based on time before check-in.
- Handling invalid input and application-specific business errors without terminating the application.
- Persisting multiple object collections between application sessions using serialization.

12. Learnings & Key Takeaways

The project provided practical experience in applying Java concepts to a real-world problem. The implementation demonstrates how classes and objects can model entities, how encapsulation helps manage state, and how collections can store dynamic records. It also provided experience with custom exception handling, file I/O, serialization, date calculations, validation, and organizing a multi-class Java application.

13. Future Enhancements

- Replace serialized files with MySQL or SQLite using JDBC.
- Support multi-user concurrent access.
- Add online payment integration and email confirmations.
- Add room housekeeping tracking, loyalty points, and guest reviews.
- Develop a GUI using Swing or JavaFX.
- Develop a web-based or mobile interface.
- Add revenue reports, staff management, service-request tracking, and inventory management.

14. References

- CSE2006 – Programming in Java course material.
- Herbert Schildt, Java: The Complete Reference.
- Oracle Java Tutorials – Collections Framework.
- Oracle Java Tutorials – Date and Time.
- Oracle Java Tutorials – Exception Handling.